Multi-Linear Regression

Rui Zhou

目录


```
{r warning=FALSE} library(dplyr)
```

The `df` contains information about salary, beginning salary, education year, job category, previous working experience and so on. The relationship between salary and beginning salary, education, to name a few, is of interest.

```
{r} df = readxl::read_excel("data02-01.xlsx", sheet = 1)
```

1 Pre-Inspection

1.1 linear relationship

Use `cor()` to carry the correlation test, the default method of which is `method = 'pearson'`. The result will be presented as a correlation matrix.

However, obviously the correlation matrix won't give you information about p-value (or say, significance).

Good thing is that the check of linear relationship is the first and rough step for further regression, maybe other more information (about p-value or CIs) is unnecessary. Another good thing is that the `cor()` is from base R, no extra requirement to library packages.

```
{r comment=''} df |> select(salary,educ,salbegin,jobtime,prevexp) |> cor()
```

Well, it's clumsy to compress the target data (variables) into a matrix to take the test and get the correlation matrix. Just fine to know a bit about that. `{r comment=''} # first create an empty matrix, primarily filled with NA m = matrix(nrow=length(salary),ncol=5) m[,1]=salary m[,2]=educ m[,3] = salbegin m[,4]=jobtime  m[,5]=prevexp #check the correlation matrix cor(m)`

Additionally, I'd like to use `bruceR::Corr()` for the correlation matrix, which is visible for both in amount and in significance. Moreover, it's fancy!

For simplicity, the two decimals by default is well enough. If more accuracy is needed, you can set the parameter `digits =`.

```
{r comment=''} df |> select(salary,educ,salbegin,jobtime,prevexp) |> bruceR::Corr(digits
= 2)
```

1.2 multicollinearity

Use `car::vif(model)` and get the result directly.

For the convenience of narration, check of multicollinearity is presented here. Most of the time, the check is a "post-hoc" one, after obtaining the fitted model.

VIF less than 2 is ideal, and less than 5 is the bottom line.

```
{r comment=''} fit = lm(salary ~ educ + salbegin + jobtime + prevexp,data = df) car::vif(fit)
```

2 MLR

2.1 basic summary

Firstly, use `lm()` to get the linear model.

Subsequently, use `summary()` to get the detailed result.

Period. `{r comment=''} fit = lm(salary ~ educ + salbegin + jobtime + prevexp,data = df) summary(fit)`

It's noteworthy that `bruceR::print_table(model)` can spring up the regression result as a decent table, and can be exported to a `.doc` file.

However, fine with `summary()`, decency doesn't matter much here, and the `summary()` can give more information about the model itself that is essential, such F value and multiple R squared.

2.2 coefficients

Use `coefficients(model)` from base R to see the coefficients for each IV.

Furthermore, use `confint(model,level = 0.95)` from base R to see the CIs (95% by default) for each coefficients.

`{r} coefficients(fit) confint(fit, level=0.95)`

Multiple R squared is reported through `summary()`.

By definition, Multiple R squared is defined as the square of the correlation between estimated (fitted) value and real values. `{r comment=''} cor(salary,fit$fitted.values)`

2.3 model comparison

Compare the model with the "zero" model, formally called the null model. The null model has no IV but with intercept, which is the mean. The null model uses the mean to predict the data or explain the variation of values.

Get the null model by formula `DV~1`.

`{r comment=''} fit2 = lm(salary ~ 1 )`

Then use `anova(model1,model2)` to compare the two models.

`{r comment=''} anova(fit,fit2)`

If we think further, the F test for the MLR model is to test with null hypothesis that all the coefficients of all the IVs are zero. Therefore, the model comparison by essence coincides with the F test! Obviously, the F value "copies" that in the `summary()` result.

3 Standard MLR

Transform the formula with each variable embedded into the function `scale()`, no other adjustments!

Under standard MLR, the coefficients are standardized ones. `{r comment=''} fit_std = lm(scale(salary) ~ scale(educ) + scale(salbegin) + scale(jobtime) + scale(prevexp),data = df)`

`{r comment=''} summary(fit_std)`

3.1 (semi-) partial for x

(Semi-) partial correlation indicates the variance accounted for by the certain IV specifically, which is especially intriguing and important in standard MlR.

Use `pcor()` from the package `ppcor` to compute the partial correlation for IVs.

Use `spcor()` from the package `ppcor` to compute the semi-partial correlation for IVs.

Throw the regressed variables (both IVs & DV) into the functions above, and period.

The results will include estimated value of correlation and its p-value. However, the drawback is that the returned matrix does not include rownames and colnames, making it reading-unfriendly. However, we only care about the (semi-) partial correlation between DV and all IVs, which means only the very one row or column is enough.

Therefore, DV goes first! Only check the first row or column! "'{r comment='’} # partial correlation df |> select(salary,educ,salbegin,jobtime,prevexp) |> ppcor::pcor()

4 part or semi-partial correlation

df |> select(salary,educ,salbegin,jobtime,prevexp) |> ppcor::spcor()

```
# Goodness of Fit
## fitted values & real values
```{r comment=''}
fitted_y = fitted(fit) # predicted values
plot(df$salary,fitted_y,'p')
```
```

4.1 residual

If the model fits well, then the residual should follow a normal distribution.

Fitted model's residuals can be obtained through `residuals(model)` or `model$residuals`. This works the same for other attributes of a linear model, such as coefficients. `{r comment=''} hist(residuals(fit)) # hist(fit$residuals)`

4.2 plot more

Use `plot(model)` to get multiple plots reflecting the goodness of fit from many perspectives.

Specify parameter `which = vector_of_range` to designate the plots to show, 6 the most. `which = 1:4` by default.

Following the order, the six plots are:

- Residuals & Fitted Values
  - test the homoscedascitiy assumption
- QQ Plot for Residuals
  - test the Gaussian assumption
- Square Root of Standardized Residuals & Fitted Value
  - similar to the first one
- Cook's Distance
  - check outliers
- Standardized Residuals & Leverage
  - See if influential points have huge residual, if so, possibly outlier!
- Cook's Distance & Leverage
  - Large in both, possibly outlier! `{r comment=''} plot(fit, which=1:6)`

There's another way to do the model checking. Use `performance::check_model(model)`.

The default option is to check all the "vif", "qq", "normality", "linearity", "ncv", "homogeneity", "outliers", "reqq", "pp_check", "binned_residuals" and "overdispersion", which can be specified through `check = specifying_vector`. Here I'll take the check of outliers as an example. `{r comment=''} performance::check_model(fit,check = 'outliers')`

Do not forget to `detach()` the attached data. `{r comment=''} detach(df)`

5 Appendix

5.1 Attach and Detach

We can use the function `attach()` to use variables in the database conveniently. After `attach()`, column variables can be used directly without `$`. (R will do the searching work behind for you!)

However, sometimes names of the variables will conflict with each other and cause problems. Not recommended from my perspective, because you'll not know what database the variable is from, especially when you're trying to check the codes. Combined with tube-friendly and formula-pattern genre, this won't be any more trouble.

```
{r} attach(df)
```

After attaching a database, remember to `detach()` that finally, in case of conflicts with other variables.

5.2 MLR

```
{r} heart = read.csv('heart.csv')
```

5.2.1 Fit the Model

```
{r} mfit = lm(scale(heart.disease)~scale(biking)+scale(smoking),           data = heart)
```

5.2.2 Summary the Model

```
{r} summary(mfit)
```

It's predicted that if `biking` changes for one standard deviation, then `heart.disease` is to change for -0.940 standard deviation, and that if `smoking` changes for one standard deviation, then `heart.disease` is to change for 0.323 standard deviation. Meanwhile, coefficients of both `biking` and `smoking` are highly significant with $p < .001$.

5.2.3 Goodness of Fit

After qualitative analysis, now move on to quantitative works.

```
{r comment=''} result = summary(mfit) result$adj.r.squared
```

Adjusted R^2 of 0.9795 indicates a highly awesome goodness of fit and its ability to explain the variation of the data!

```
{r comment=''} cor(x = fitted(mfit), y = heart$heart.disease)
```

The correlation between the fitted values and the observed (real) values is as high as 0.9897563, which is exactly the square root of R^2. Jointly speaking, the model has great credibility in prediction.

5.2.4 VIF

```
{r comment=''} car::vif(mfit)
```

VIFs of `biking` and `smoking` are 1.000229 and 1.000229 respectively, far less than 5, showing that there should be no worries about multicollinearity.

5.2.5 Plots of Residuals

```
{r} par(mfrow = c(1,2)) plot(mfit,which = c(1,3))
```

From the first plot, it's shown that there's no obvious tendency between the fitted values and the residuals.

```
{r} par(mfrow = c(1,2)) plot(mfit,which = 2) hist(mfit$residuals)
```

In QQ plot, the scatter points of residuals closely center around the line offered by normal distribution. In the histogram of residuals, a clear prototype of normal distribution is depicted. What's more, move on from qualitative analysis to the quantitative.

```
{r comment=''} shapiro.test(mfit$residuals) mfit$residuals |> mean()
```

From the Shapiro test, $W = 0.9969963, p = 0.4935143$, indicating that the residuals from the fitted model follow a normal distribution. Moreover, the mean is computed to be (extremely close to) zero.

Therefore, we can safely say that the residuals of the fitted model do follow a normal distribution.

Based on what we've discussed, we can conclude confidently that the model fits the data well!